VIT®

BHOPAL

Student Name - MOHIT YADAV

Student Regn. No. - 25BAI11342

Semester – FALL SEMESTER

Year : 2025 - 26

Slot: F11 + F12

Course Code – CSE2006

Subject – PROGRAMMING IN JAVA

Faculty Name- Dr. RIZWAN UR REHMAN
School of Computing Science
Engineering and Artificial Intelligence (SCAI) VIT Bhopal University

Project Report

PROJECT TITLE

SMART MULTI-CITY ROUTE PLANNER

1. Introduction

- Transportation networks contain many stations connected through different routes. When a user wants to travel from one station to another, there may be several possible paths.
- Manually comparing all possible routes becomes difficult as the number of stations and connections increases.
- Computer algorithms can solve this problem efficiently by representing the transportation network as a graph.
- In this project, each station is represented as a **vertex**, while a route between two stations is represented as an **edge**. The distance between stations is stored as the weight of the edge.
- The project uses **Dijkstra's Shortest Path Algorithm** to determine the shortest route between a source and destination.
- The application is implemented in Java and provides a simple command-line interface for interacting with the transportation network.

---

2. PROBLEM STATEMENT

In a transportation network, multiple stations may be connected through several routes.

For example, a passenger travelling from Station A to Station D may have multiple possible paths:

A → B → D

A → C → D

A → B → C → D

Each route can have a different total distance.

The main problem is:

How can we efficiently determine the shortest route between two stations in a multi-city transportation network?

The project solves this problem by:

1. Representing stations as graph vertices.
2. Representing routes as weighted edges.
3. Storing the graph using an adjacency list.
4. Applying Dijkstra's Shortest Path Algorithm.
5. Calculating the total route distance.
6. Estimating travel time and fare.

2. Objectives

- To create a multi-city transportation network.
- To represent transportation stations using graph vertices.
- To represent routes using weighted edges.
- To implement Dijkstra's Shortest Path Algorithm.
- To find the shortest route between two stations.
- To calculate total route distance.
- To estimate travel time.
- To calculate estimated fare.
- To display available transportation details.
- To demonstrate Java Object-Oriented Programming.
- To use Java Collections Framework.

---

4. SCOPE OF THE PROJECT

The current project focuses on developing an academic route-planning system using Java.

The system supports the following cities:

- Bhopal
- Indore
- Ujjain
- Sehore

The application provides:

- City selection.
- Station listing.
- Route listing.
- Shortest route calculation.
- Distance calculation.
- Travel-time estimation.
- Fare estimation.
- Transportation information.
- Input validation.

The current version is primarily designed for learning and academic demonstration.

It does not currently provide:

- Real-time GPS.
- Real-time traffic.
- Live transportation schedules.
- Official transportation fares.
- Interactive maps.
- Online database connectivity.

5. EXISTING SYSTEM

Traditional route planning can involve manually checking different routes between two locations.

For small transportation networks, this may be possible, but as the number of stations and connections increases, manual route comparison becomes difficult.

Problems with the Existing Approach

- Manual route comparison.
- Time-consuming process.
- Difficult to handle large networks.
- Higher possibility of human error.
- No automatic shortest-path calculation.
- Difficult to calculate route metrics quickly.

6. PROPOSED SYSTEM

- The proposed system provides an automated solution for finding the shortest route.
- The transportation network is represented as a weighted graph.
- The system performs the following operations:

```
User
 ↓
Select City
 ↓
View Stations / Routes
 ↓
Enter Source
 ↓
Enter Destination
 ↓
Validate Input
 ↓
Create/Access Graph
 ↓
Dijkstra's Algorithm
 ↓
Find Shortest Path
 ↓
Calculate Distance
 ↓
Calculate Time and Fare
 ↓
Display Result
```

This approach reduces the effort required to manually compare different routes.

7. REQUIREMENTS ANALYSIS

7.1 Functional Requirements

The system should be able to:

1. Display available cities.
2. Allow the user to select a city.
3. Display stations.
4. Display available routes.
5. Accept source station.
6. Accept destination station.
7. Validate station input.
8. Find the shortest route.
9. Calculate total distance.
10. Calculate estimated travel time.
11. Calculate estimated fare.
12. Display transportation details.
13. Handle invalid input.
14. Return to the city-selection menu.
15. Exit the application.

7.2 Non-Functional Requirements

The system should:

- Be easy to use.
- Provide fast results.
- Use minimum system resources.
- Produce reliable shortest-path results.
- Be maintainable.
- Be portable across systems supporting Java.
- Have a simple command-line interface.

7.3 Hardware Requirements

Basic requirements include:

- Computer or laptop.
- Minimum 4 GB RAM recommended.
- Keyboard and display.
- Processor capable of running Java applications.

7.4 Software Requirements

- Java Development Kit (JDK).
- Command Prompt / PowerShell / Terminal.
- Java-compatible IDE such as:
    - Visual Studio Code
    - IntelliJ IDEA
    - Eclipse

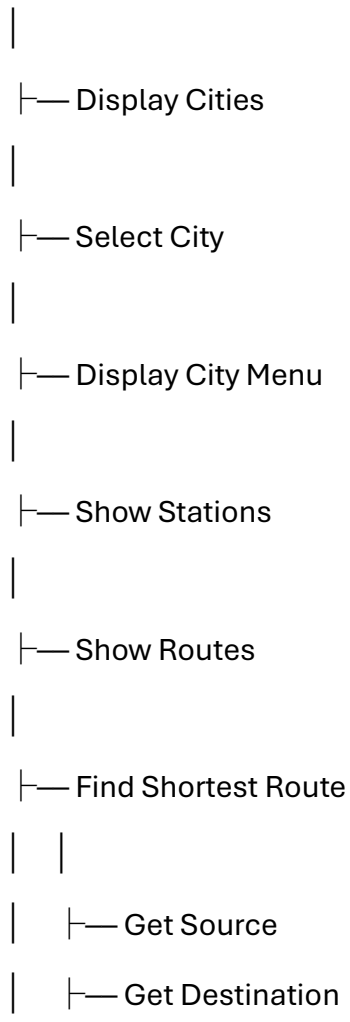
8. SYSTEM DESIGN

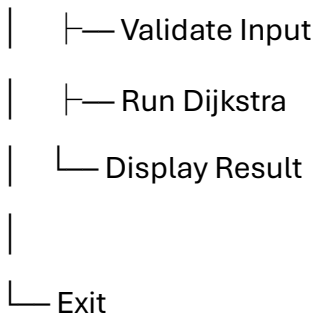
8.1 Top-Down Design

The project follows a top-down approach.

The major components are:

main()





8.2 Major Functions

- **Network Creation**
  Creates stations and routes.
- **Station Management**
  Stores and displays station information.
- **Route Management**
  Creates connections between stations.
- **City Management**
  Stores and manages individual city networks.
- **Shortest Path**
  Uses Dijkstra's algorithm to find the shortest route.
- **Input Validation**
  Checks whether the user has entered valid data.

9. GRAPH REPRESENTATION

The transportation system is represented using a **weighted graph**.

A graph consists of:

- Vertices
- Edges
- Weights

In this project:

Vertex = Station

Edge = Route

Weight = Distance

Possible routes:

A → B → D

Distance:

5 + 4 = 9 km

Another route:

A → C → D

Distance:

8 + 6 = 14 km

Therefore:

Shortest Route = A → B → D

Shortest Distance = 9 km

10. DIJKSTRA'S ALGORITHM

Dijkstra's algorithm is a shortest-path algorithm used for finding the minimum distance between vertices in a weighted graph.

The algorithm works with non-negative edge weights.

10.1 Steps

1. Select the source station.
2. Set its distance to 0.
3. Set all other station distances to infinity.
4. Insert the source into a PriorityQueue.
5. Select the station with the smallest known distance.
6. Examine its neighboring stations.
7. Calculate new possible distances.
8. Update the distance if a shorter route is found.
9. Store the previous station.
10. Continue until the destination is reached.
11. Reconstruct the shortest path.

10.2 PriorityQueue

A PriorityQueue is used because the algorithm repeatedly needs to process the station with the smallest current distance.

Conceptually:

PriorityQueue

↓

Minimum Distance Station

↓

Process Neighbors

↓

Update Distances

↓

PriorityQueue

11. OBJECT-ORIENTED DESIGN

The project demonstrates several Object-Oriented Programming concepts.

11.1 Classes and Objects

Classes are used to define the structure and behavior of different project components.

Objects represent individual instances of these classes.

11.2 Encapsulation

Data members can be kept private and accessed through appropriate methods.

This helps protect the internal state of objects.

11.3 Inheritance

Transportation classes can inherit common properties and methods from a parent transportation class.

For example:

Transportation

|

├── CityBus

|

└── Metro

11.4 Abstraction

Abstract classes or interfaces can define common behavior without exposing unnecessary implementation details.

11.5 Polymorphism

A parent transportation type can refer to different child objects such as Bus and Metro.

The same method can produce different behavior depending on the object.

11.6 Method Overriding

Child classes can override methods defined in the parent class.

For example, Bus and Metro can implement different fare calculations.

11.7 Interface

Interfaces can be used to define common behavior that different classes must implement.

11.8 Comparable

Comparable can be used to compare objects based on distance.

This is particularly useful with PriorityQueue.

11.9 Exception Handling

Exception handling is used to prevent the application from crashing when the user enters invalid input.

12. DATA STRUCTURES USED

The project uses several Java data structures.

12.1 HashMap

HashMap is used to store mappings such as station information and graph connections.

Example:

Station ID → Station Information

12.2 ArrayList

ArrayList is used for storing collections of routes or connections.

12.3 HashSet

HashSet can be used for storing unique stations or tracking visited elements.

12.4 PriorityQueue

PriorityQueue is used by Dijkstra's algorithm to select the station with the smallest current distance.

12.5 Graph

The transportation network itself is represented as a weighted graph.

---

13. TRANSPORTATION AND FARE MODEL

The project includes simulated transportation modes such as:

- City Bus
- Metro

The fare and travel-time calculations are designed only for academic demonstration.

13.1 City Bus

The simulated fare formula is:

Fare = 10 + (Distance × 2)

Travel time:

Time = Distance × 3 minutes

13.2 Metro

The simulated fare formula is:

Fare = 15 + (Distance × 1.5)

Travel time:

Time = Distance × 2 minutes

Example

For a distance of 10 km:

Bus

Fare = 10 + (10 × 2)

= ₹30

Time = 10 × 3

= 30 minutes

Metro

Fare = 15 + (10 × 1.5)

= ₹30

Time = 10 × 2

= 20 minutes

Note: These are simulated values created for academic purposes and are not official fares or travel schedules.

14. SYSTEM WORKFLOW

The system follows the following workflow:

Step 1: Start

The Java program starts execution.

Step 2: Create Network

The transportation network containing stations and routes is created.

Step 3: Display Cities

The available cities are displayed.

Step 4: Select City

The user selects one city.

Step 5: Display Menu

The city-specific menu is displayed.

Step 6: Select Operation

The user can:

- View stations.
- View routes.
- Find shortest route.
- Return.
- Exit.

Step 7: Enter Source and Destination

The user enters the starting and ending stations.

Step 8: Validate Input

The system checks whether the stations are valid.

Step 9: Run Dijkstra

Dijkstra's algorithm calculates the shortest path.

Step 10: Display Result

The shortest route and calculated information are displayed.

15. IMPLEMENTATION DETAILS

The project is implemented in Java.

The main source file is:

main.java

Project structure:

Smart-Multi-City-Route-Planner/

├── main.java

├── README.md

└── statement.md

15.1 Main Components

main()

Controls the overall execution of the application.

Network Creation

Creates stations and their connections.

addStation()

Adds stations to the network.

connect()

Creates a connection between two stations.

showCities()

Displays available cities.

showStations()

Displays stations in the selected city.

showRoutes()

Displays available routes.

findRoute()

Accepts source and destination.

shortestPath()

Implements Dijkstra's algorithm.

belongsToCity()

Validates whether a station belongs to the selected city.

16. TESTING

Testing was performed to ensure that the application behaves correctly under different conditions.

Test Case 1 — Valid City

Input: Valid city number.

Expected Result: The selected city's menu should be displayed.

Status: Passed.

Test Case 2 — Show Stations

Input: Select "Show Stations".

Expected Result: Stations belonging to the selected city should be displayed.

Status: Passed.

Test Case 3 — Show Routes

Input: Select "Show Routes".

Expected Result: Available routes should be displayed.

Status: Passed.

Test Case 4 — Valid Route

Input: Valid source and destination stations.

Expected Result: Shortest route and distance should be displayed.

Status: Passed.

Test Case 5 — Invalid Station

Input: Station that does not exist.

Expected Result: Appropriate error message.

Status: Passed.

Test Case 6 — Invalid City

Input: Invalid city number.

Expected Result: Error message should be displayed.

Status: Passed.

Test Case 7 — Invalid Menu Choice

Input: Number outside the menu range.

Expected Result: Invalid-choice message.

Status: Passed.

Test Case 8 — Invalid Data Type

Input:

abc

where a number is expected.

Expected Result: Input should be handled safely.

Status: Passed.

Test Case 9 — Same Source and Destination

Input: Same station for source and destination.

Expected Result: The system should handle the condition appropriately.

Status: Passed.

Test Case 10 — Exit

Input: Exit option.

Expected Result: Program should terminate.

Status: Passed.

17. COMPLEXITY ANALYSIS

For Dijkstra's algorithm using an adjacency list and a PriorityQueue, the approximate time complexity is:

O((V + E) log V)

Where:

- V = Number of vertices/stations.
- E = Number of edges/routes.

The space complexity is approximately:

O(V + E)

This includes:

- Graph storage.
- Distance information.
- Previous-node information.
- Visited information.
- PriorityQueue.

The adjacency-list representation is memory-efficient for sparse transportation networks.

18. RESULTS AND EXPECTED OUTPUT

The system successfully demonstrates the process of selecting a city and finding the shortest route between two stations.

A typical output may look like:

==========

SMART MULTI-CITY ROUTE PLANNER

==========

1. Bhopal

2. Indore

3. Ujjain

4. Sehore

5. Exit

Enter your choice: 1

After selecting a city:

1. Show Stations

2. Show Routes

3. Find Shortest Route

4. Return

5. Exit

For a route search:

Enter source station: A

Enter destination station: D

Example result:

Shortest Route:

A → B → D

Total Distance: 9 km

Estimated Time: 27 minutes

Estimated Fare: ₹28

The exact output depends on the stations, connections, distances, and transportation information defined in the Java program.

19. LIMITATIONS

The current implementation has some limitations.

1. The application uses a command-line interface.
2. Transportation data is predefined.
3. The system does not use real-time traffic information.
4. GPS integration is not currently available.
5. Interactive maps are not provided.
6. No database is connected.
7. Fare values are simulated.
8. Travel times are simulated.
9. Live transportation schedules are not available.
10. The current project can be expanded with more inter-city connections.

20. FUTURE ENHANCEMENTS

The project can be improved significantly in the future.

20.1 More Cities

Additional cities and stations can be added.

20.2 Inter-City Routes

The system can provide direct route planning between different cities.

For example:

Bhopal → Sehore → Indore

20.3 GPS Integration

GPS can be integrated to automatically determine the user's current location.

20.4 Interactive Maps

A map interface can be added to visually display stations and routes.

20.5 Real-Time Traffic

Traffic information can be integrated to calculate more realistic travel times.

20.6 Multiple Route Options

The system could display:

- Shortest route.
- Cheapest route.
- Fastest route.

20.7 Graphical User Interface

A GUI can be created using JavaFX or Swing.

20.8 Web Application

The system can be converted into a web-based route planner.

20.9 Database

A database can store:

- Stations.
- Routes.
- Distances.
- Fares.
- Transportation schedules.

20.10 Mobile Application

The system could eventually be developed as an Android application.

21. APPLICATIONS

The concept developed in this project can be applied to:

- Public transportation.
- Metro route planning.
- Railway route planning.
- Logistics.
- Delivery systems.

- Fleet management.
- Emergency route planning.
- Navigation systems.
- Smart-city transportation systems.
- Educational demonstrations of graph algorithms.

22. LEARNING OUTCOMES

Through this project, the following concepts were practically implemented and understood:

- Java programming.
- Object-Oriented Programming.
- Classes and objects.
- Encapsulation.
- Inheritance.
- Abstraction.
- Polymorphism.
- Interfaces.
- Method overriding.
- Exception handling.
- Java Collections Framework.
- HashMap.
- ArrayList.
- HashSet.
- PriorityQueue.
- Graph representation.
- Weighted graphs.
- Adjacency lists.
- Dijkstra's algorithm.
- Shortest-path problems.

- Time complexity.
- Space complexity.
- Input validation.
- Software design.
-

23. CONCLUSION

- The **Smart Multi-City Route Planner** demonstrates the practical application of Java programming, Object-Oriented Programming, graph data structures, and shortest-path algorithms.
- The transportation network is represented as a weighted graph where stations are vertices and routes are edges. Dijkstra's Shortest Path Algorithm is used to calculate the minimum-distance route between two stations.
- The system supports four cities — **Bhopal, Indore, Ujjain, and Sehore** — and provides features such as station display, route display, shortest-route calculation, distance calculation, travel-time estimation, and fare estimation.
- The project also demonstrates important Java concepts including inheritance, abstraction, polymorphism, interfaces, exception handling, `HashMap`, `ArrayList`, `HashSet`, `PriorityQueue`, and `Comparable`.
- Although the current system is based on simulated transportation data and a command-line interface, it provides a strong foundation for developing a more advanced route-planning application with GPS, maps, real-time traffic, databases, and mobile or web interfaces.
- Overall, the project successfully connects theoretical concepts from **Data Structures, Algorithms, Graph Theory, and Java Programming** with a practical real-world problem.

24. REFERENCES

The following resources and concepts were used as references during the development of the project:

1. Java Programming concepts and Java Collections Framework.
2. Object-Oriented Programming principles.
3. Graph Theory and Weighted Graph concepts.
4. Dijkstra's Shortest Path Algorithm.
5. Data Structures and Algorithms course material.
6. Java PriorityQueue and Comparable concepts.
7. Java exception-handling concepts.

25. APPENDIX

A. Project Summary

Item	Details
Project Name	Smart Multi-City Route Planner
Programming Language	Java
Interface	Command Line
Main Algorithm	Dijkstra's Algorithm
Graph Type	Weighted Graph
Graph Representation	Adjacency List
Cities	Bhopal, Indore, Ujjain, Sehore
Main Data Structures	HashMap, ArrayList, HashSet, PriorityQueue
Main Concepts	OOP, DSA, Graph Theory
Purpose	Academic Route Planning System

B. Project Structure

```
Smart-Multi-City-Route-Planner/
│
├── main.java
├── README.md
└── statement.md
```

C. Execution Commands

Compile the Java program:

```
javac main.java
```

Run the program:

```
java main
```

Check Java version:

```
java --version
```

Check Java compiler:

```
javac --version
```

VIT®

BHOPAL